

PROGRAMMING · BEGINNER

JavaScript Foundations

Make the web respond to your ideas.



A free path from JavaScript fundamentals to browser applications and asynchronous code.



6-8 weeks



4 modules



Free course

[Explore the learning experience →](#)

Practical learning. Clear progression. Work you can explain.

YOUR NEXT CAPABILITY

What will you be able to do?

Learn language fundamentals, DOM interaction, modules, asynchronous programming and modern browser development through practical mini-projects.



Write modern JavaScript



Build interactive browser interfaces



Use promises and async/await



Structure code with modules



Who this is for

- Web development beginners
- Designers moving into code
- Learners preparing for React or Node.js



Your starting point

- Basic HTML and CSS are helpful

Your progression at a glance

FROM CONCEPT TO EVIDENCE



Learn it. Apply it. Explain it.

Move from guided concepts to practical exercises and a coherent piece of work. Use the course resources to revisit difficult ideas and make your reasoning visible.

THE CURRICULUM

A clear route through the subject.

Every module builds towards practical work. The course outline below is aligned with the academy's current program catalog.

1 JavaScript Language Foundations

Values, functions, control flow and data structures.

- Values and variables
- Functions
- Arrays and objects
- Errors and debugging

2 Browser and DOM

Events, forms and accessible interaction.

- DOM selection
- Events
- Forms and validation
- Accessible UI state

3 Asynchronous JavaScript

APIs, promises and resilient data loading.

- Promises
- Async and await
- Fetch and JSON
- Loading and error states

4 Modules and Capstone

Organize and ship a small browser product.

- ES modules
- Tooling basics
- Testing concepts
- Interactive web app capstone

FROM CONCEPT TO EVIDENCE



Practice with purpose

Combine events, DOM updates and asynchronous behavior in a small web application that people can use and understand.

BUILD SOMETHING THAT DEMONSTRATES YOUR LEARNING

Interactive browser application

Combine events, DOM updates and asynchronous behavior in a small web application that people can use and understand.

**Interactive interface****Documented event and data flow****Responsive project and test notes**

Tools and working methods

Browser DevTools

VS Code

Git

Tools are part of the learning workflow, not partner endorsements. Examples may evolve while preserving the learning outcomes.



Delivery

Self-paced online · 6–8 weeks

Confirm the current schedule, delivery arrangements and any prerequisites before enrolling.



Completion

A completion certificate is available after the required course work and a passing assessment.

Course completion is not a job, placement or funding guarantee.

Your next steps

- 1 Review the curriculum and your starting point.
- 2 Create your learner account and enroll for free.
- 3 Start learning, keep your evidence and track your progress.

[Open the course experience →](#)

Customer-preview edition. Interactive links open the local academy prototype. Confirm course availability, schedule and commercial terms with the academy.

INSIDE THE LEARNING EXPERIENCE

Build a predictable filter interaction

A course list should update as a learner types. The original course data must remain available so clearing the search restores every item.

Understand the concept

Keep source data separate from the rendered result. `Array.filter` creates a new selection rather than removing elements from the original array. Normalize both the query and the title before comparison. Render user-derived text with `textContent` so a query is treated as data rather than HTML.

Worked example

```
const courses = ["C Fundamentals", "JavaScript", "Full Stack"];
const query = "script".trim().toLowerCase();
const visible = courses.filter(title =>
  title.toLowerCase().includes(query)
);
output.textContent = visible.join(", ");
```

Interpret the result

The result is JavaScript. Clearing the query matches every title. A separate no-results message can explain an empty selection without altering the source array.

Knowledge check

Should a no-results search delete items from the source array?

Explanation: No. Filtering is a view of the data; clearing the query should restore the original list.

YOUR PRACTICAL ASSIGNMENT

Build evidence, not just familiarity.

Build a predictable filter interaction



Your task

- Connect the filter to an input event and show a result count.
- Add category filtering that composes with the text query.
- Restore both filters with a keyboard-accessible reset control.



Submission review criteria

- Case-insensitive matching and a useful empty state
- No HTML injection through user text
- Source data remains unchanged after repeated filtering

Submit

A working artifact or analysis, a short explanation of your decisions, and evidence that addresses each review criterion.

Reflect

Describe one failed approach, how you diagnosed it, and what you would test next. Identify limitations instead of hiding them.

Your project connection

Combine events, DOM updates and asynchronous behavior in a small web application that people can use and understand.

Use the sample assignment as a stepping stone toward the course project, not as a substitute for the full curriculum.

[Explore the worked lesson](#) →

Proposed teaching sample for curriculum and UX review. Full lesson production, instructor review and delivery scheduling remain separate work.